Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

OLYMPIC DATA ANALYSIS PROJECT

OBJECTIVE

The primary goal is to:

1. Analyze the dataset to understand trends in medal distribution.
2. Identify the top-performing countries and athletes.
3. Study the gender distribution of events and medals.
4. Visualize the data using Python.

Data Preparation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report, confus
from sklearn.ensemble import RandomForestClassifier
```

```
df = pd.read_csv("Summer-Olympic-medals-1976-to-2008.csv", encoding='latin1'
```

```
df.head()
```

	City	Year	Sport	Discipline	Event	Athlete	Gender	Coun
0	Montreal	1976.0	Aquatics	Diving	3m springboard	KÖHLER, Christa	Women	
1	Montreal	1976.0	Aquatics	Diving	3m springboard	KOSENKOV, Aleksandr	Men	
2	Montreal	1976.0	Aquatics	Diving	3m springboard	BOGGS, Philip George	Men	
3	Montreal	1976.0	Aquatics	Diving	3m springboard	CAGNOTTO, Giorgio Franco	Men	
4	Montreal	1976.0	Aquatics	Diving	10m platform	WILSON, Deborah Keplar	Women	

Next steps: Generate code with df New interactive sheet

```
df.shape
```

```
(15433, 11)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15433 entries, 0 to 15432
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   City          15316 non-null  object
 1   Year          15316 non-null  float64
 2   Sport         15316 non-null  object
 3   Discipline    15316 non-null  object
 4   Event         15316 non-null  object
 5   Athlete       15316 non-null  object
 6   Gender        15316 non-null  object
 7   Country_Code  15316 non-null  object
 8   Country       15316 non-null  object
 9   Event_gender  15316 non-null  object
 10  Medal         15316 non-null  object
dtypes: float64(1), object(10)
memory usage: 1.3+ MB
```

```
df.isnull().sum()
```

	0
City	117
Year	117
Sport	117
Discipline	117
Event	117
Athlete	117
Gender	117
Country_Code	117
Country	117
Event_gender	117
Medal	117

dtype: int64

```
df = df.dropna()
```

```
df_ml = df.drop(columns=['Athlete'])
```

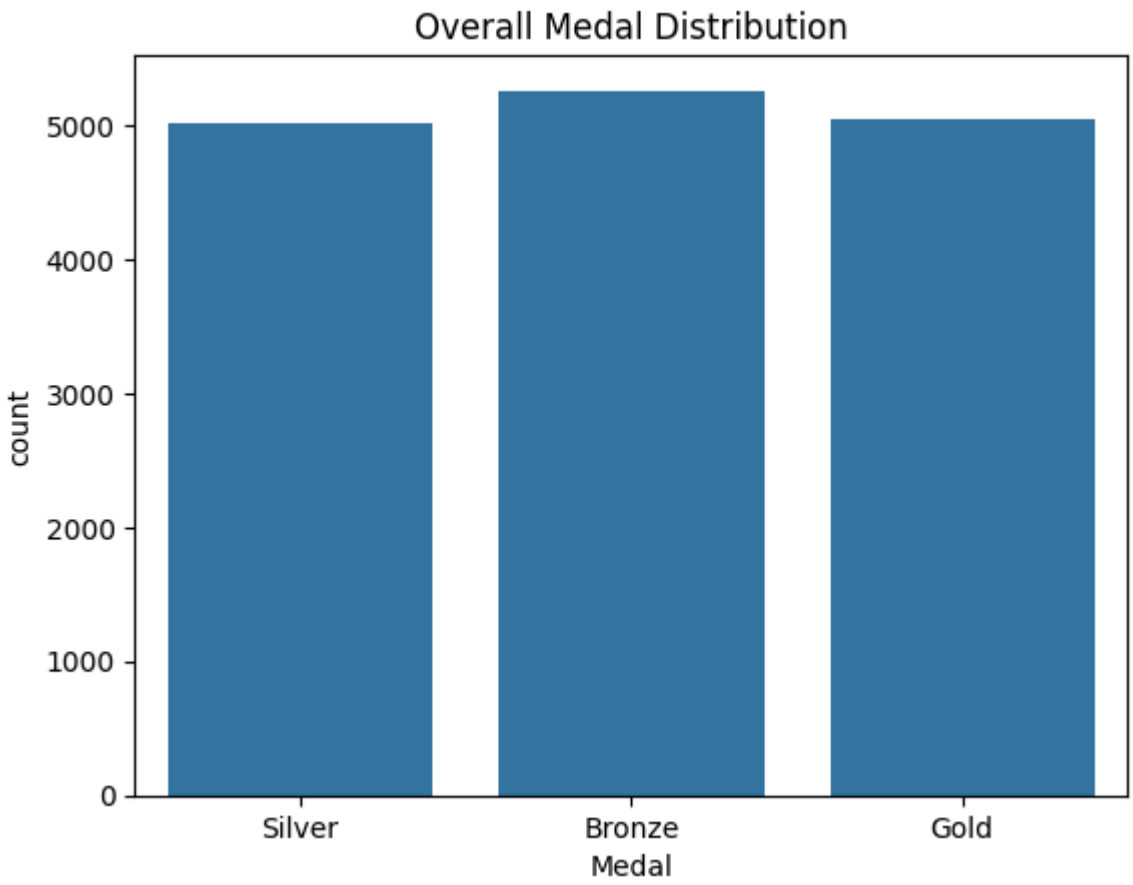
Exploratory Data Analysis (EDA)

```
df['Medal'].value_counts()
```

Medal	count
Bronze	5258
Gold	5042
Silver	5016

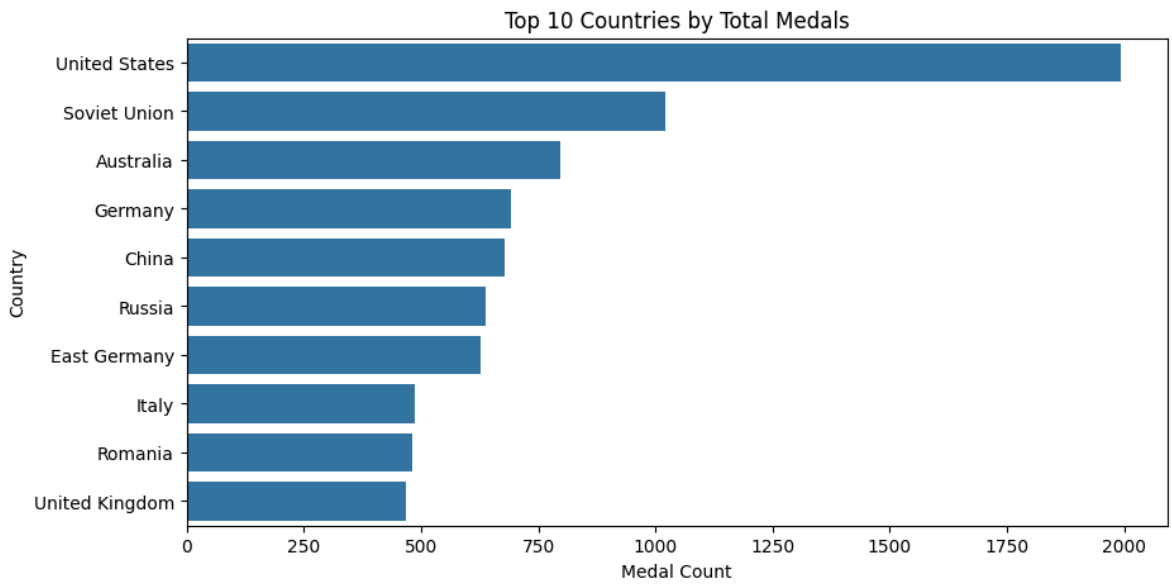
dtype: int64

```
sns.countplot(x='Medal', data=df)
plt.title("Overall Medal Distribution")
plt.show()
```



Visualizing Key Insights

```
top_countries = df['Country'].value_counts().head(10)

plt.figure(figsize=(10,5))
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title("Top 10 Countries by Total Medals")
plt.xlabel("Medal Count")
plt.show()
```

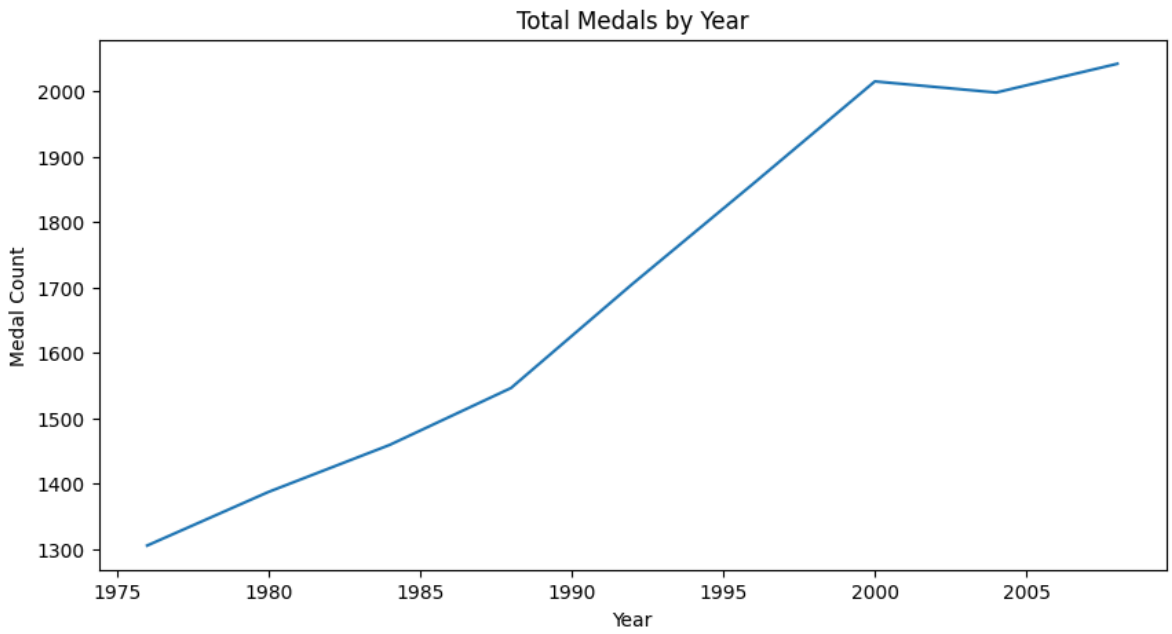


```
country_medal = df.groupby(['Country','Medal']).size().unstack().fillna(0)
country_medal.head(10)
```

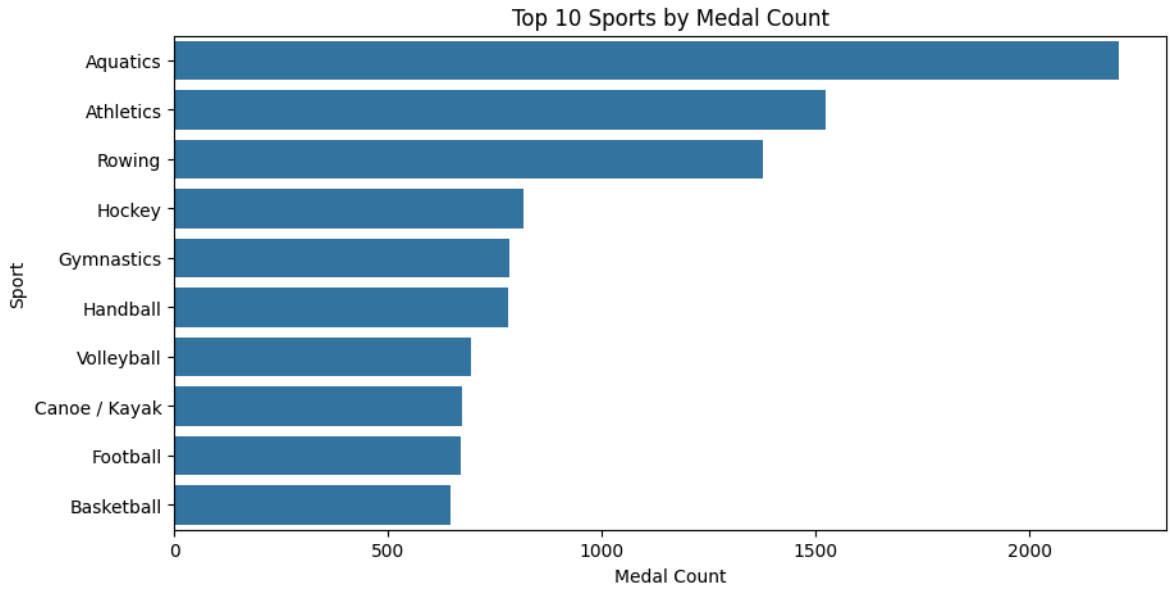
Medal Country	Bronze	Gold	Silver
Afghanistan	1.0	0.0	0.0
Algeria	8.0	4.0	2.0
Argentina	70.0	46.0	37.0
Armenia	7.0	1.0	1.0
Australia	312.0	216.0	270.0
Austria	8.0	9.0	17.0
Azerbaijan	9.0	4.0	3.0
Bahamas	3.0	7.0	9.0
Barbados	1.0	0.0	0.0
Belarus	53.0	14.0	25.0

Next steps: Generate code with `country_medal` New interactive sheet

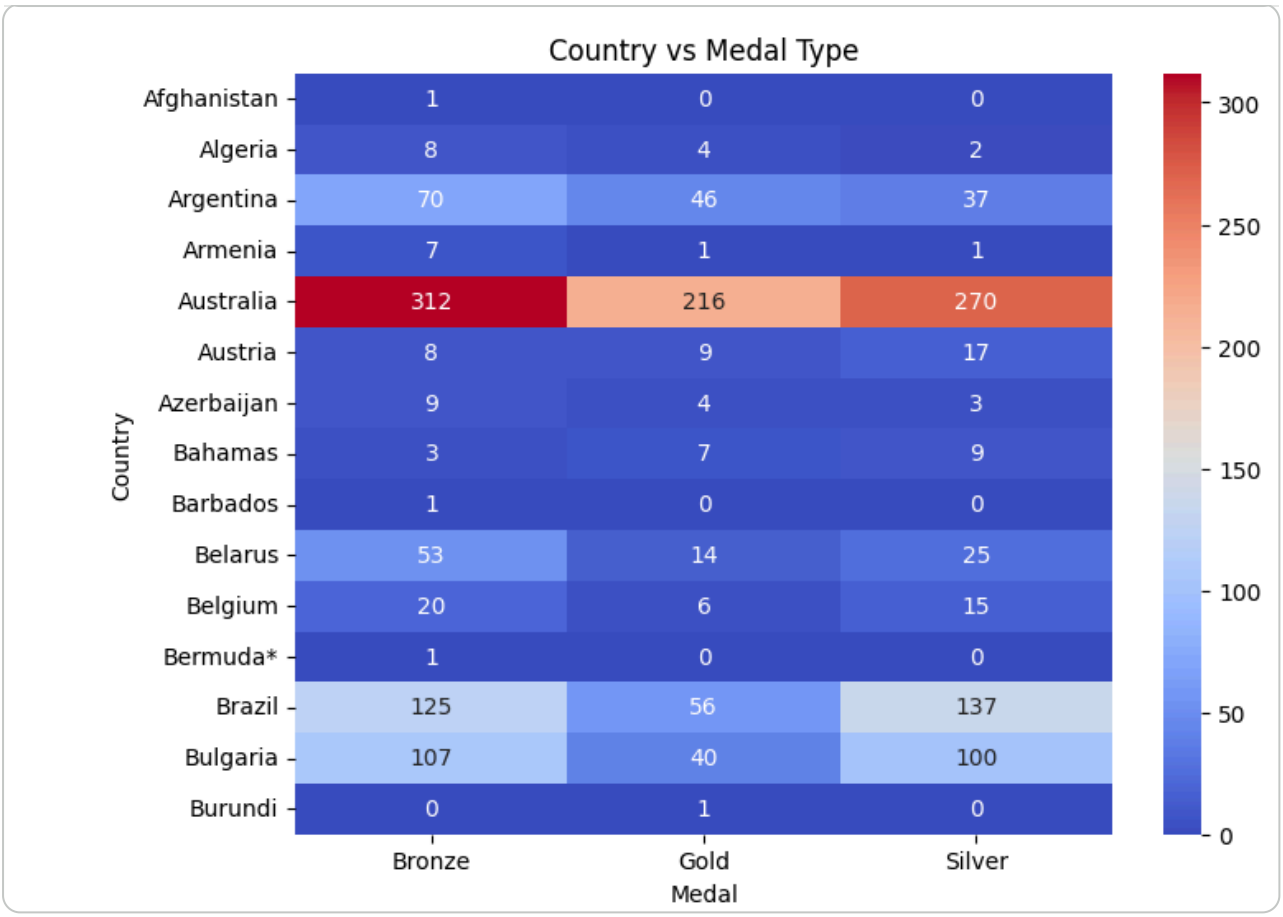
```
year_medals = df.groupby('Year').size()

plt.figure(figsize=(10,5))
sns.lineplot(x=year_medals.index, y=year_medals.values)
plt.title("Total Medals by Year")
plt.xlabel("Year")
plt.ylabel("Medal Count")
plt.show()
```




```
top_sports = df['Sport'].value_counts().head(10)

plt.figure(figsize=(10,5))
sns.barplot(x=top_sports.values, y=top_sports.index)
plt.title("Top 10 Sports by Medal Count")
plt.xlabel("Medal Count")
plt.show()
```



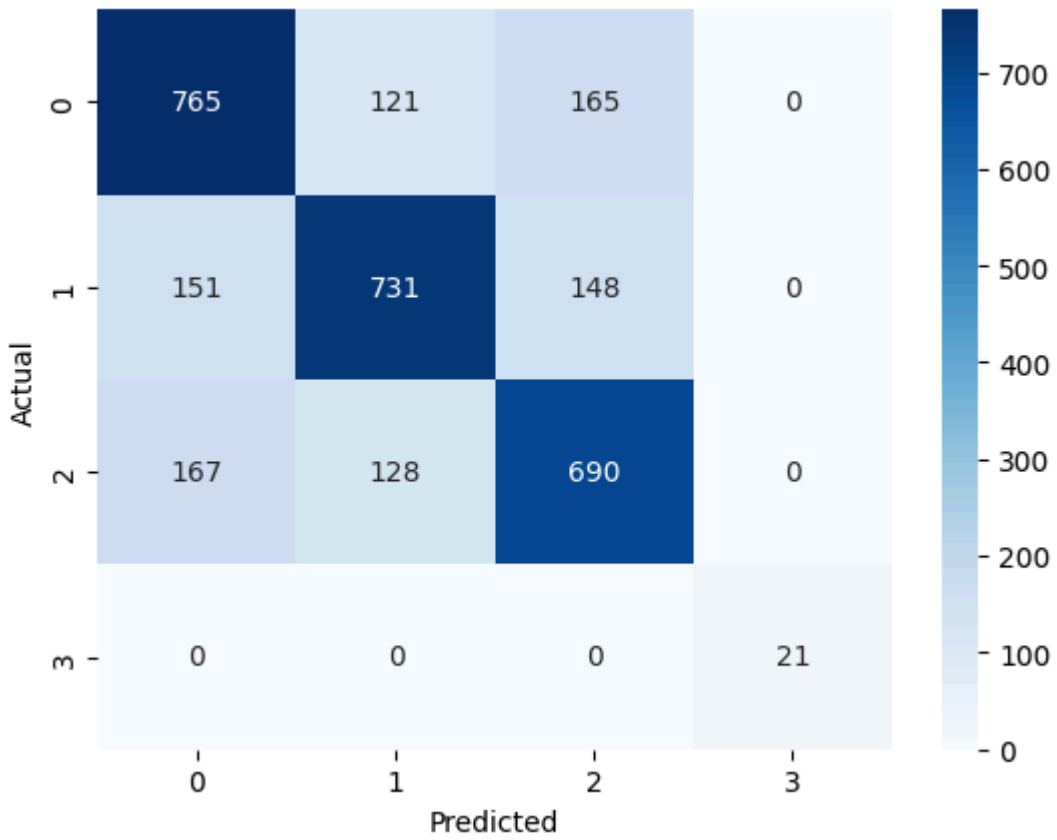

```
pivot = df.pivot_table(index='Country', columns='Medal', aggfunc='size', fil

plt.figure(figsize=(8,6))
sns.heatmap(pivot.head(15), annot=True, fmt='d', cmap='coolwarm')
plt.title("Country vs Medal Type")
plt.show()
```



Predictive Analysis

```
le = LabelEncoder()

for col in df_ml.columns:
    if df_ml[col].dtype == 'object':
        df_ml[col] = le.fit_transform(df_ml[col])
```

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
X = df_ml.drop('Medal', axis=1)
y = df_ml['Medal']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)
```

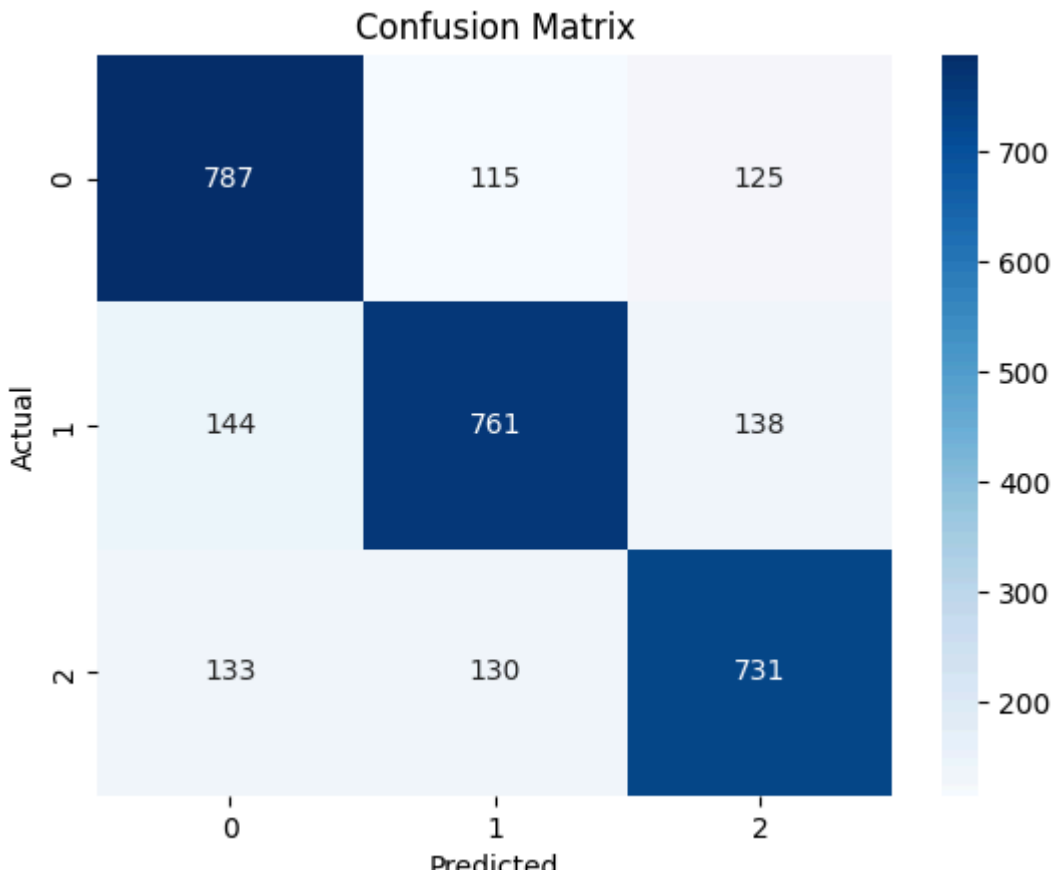
```
RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.743798955613577
              precision    recall  f1-score   support

           0       0.74      0.77      0.75      1027
           1       0.76      0.73      0.74      1043
```

```
           2       0.74      0.74      0.74       994

    accuracy                           0.74      3064
   macro avg       0.74      0.74      0.74      3064
weighted avg       0.74      0.74      0.74      3064
```

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.